doit

Kubernetes Scaling for Savings and Stability

Beyond HPA and VPA

Conf42 Observability, Sep. 2026

<<

Experts

Google Dev Expert

Joshua Fox

joshua@doit.com

Senior Cloud Architect

DoiT

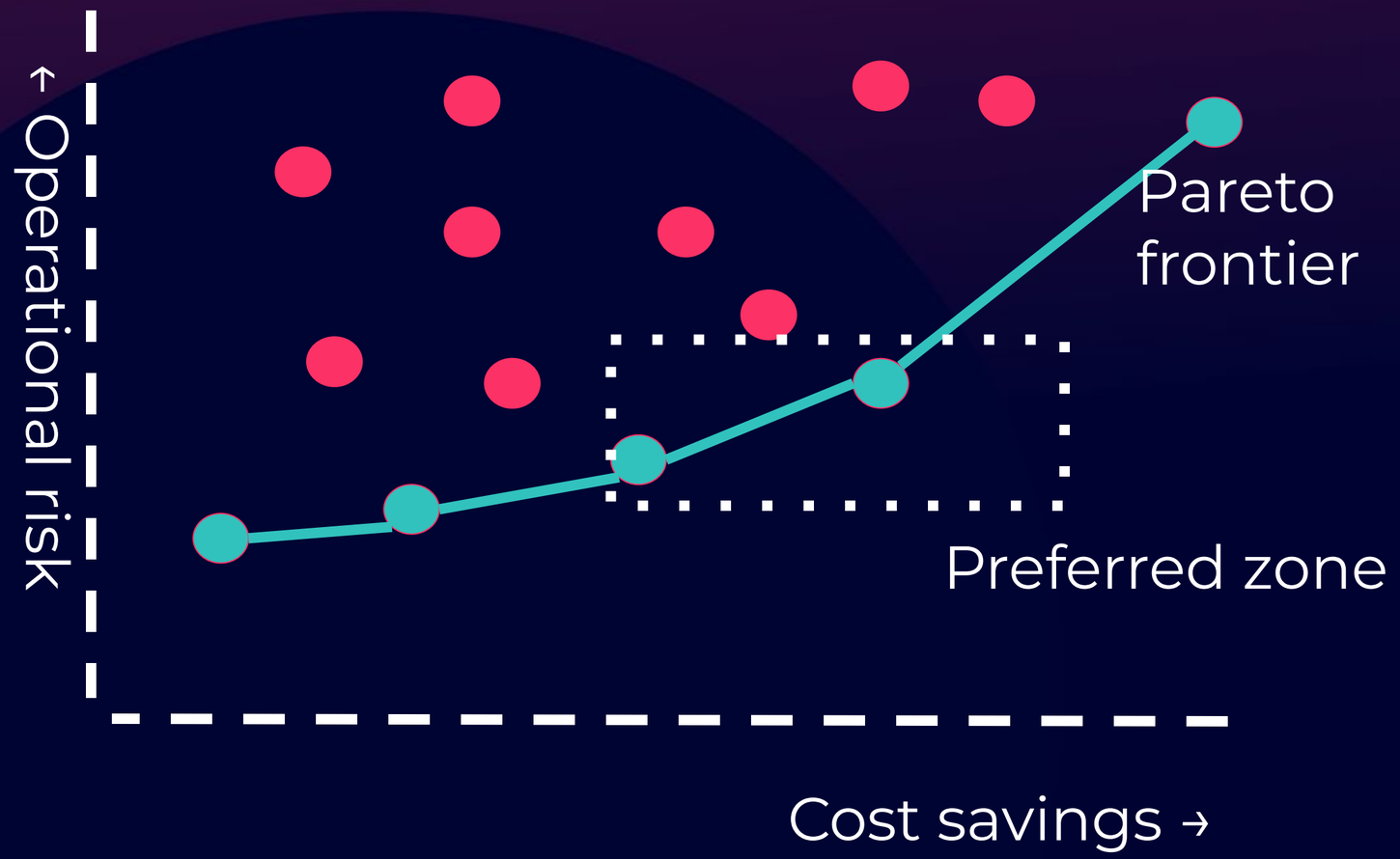
aws CERTIFIED Solutions Architect Professional

Google Cloud Certified PROFESSIONAL CLOUD ARCHITECT

Microsoft CERTIFIED

doit

Cost Savings is not Enough



Autoscaling

- Node and Pod (we’ll focus on Pod Autoscaling)
- Horizontal and Vertical

Aspects of Pod Scaling

Right-size

Tune CPU and memory requests/ limits

Scale

Add or remove replicas based on demand and business/ application signals.

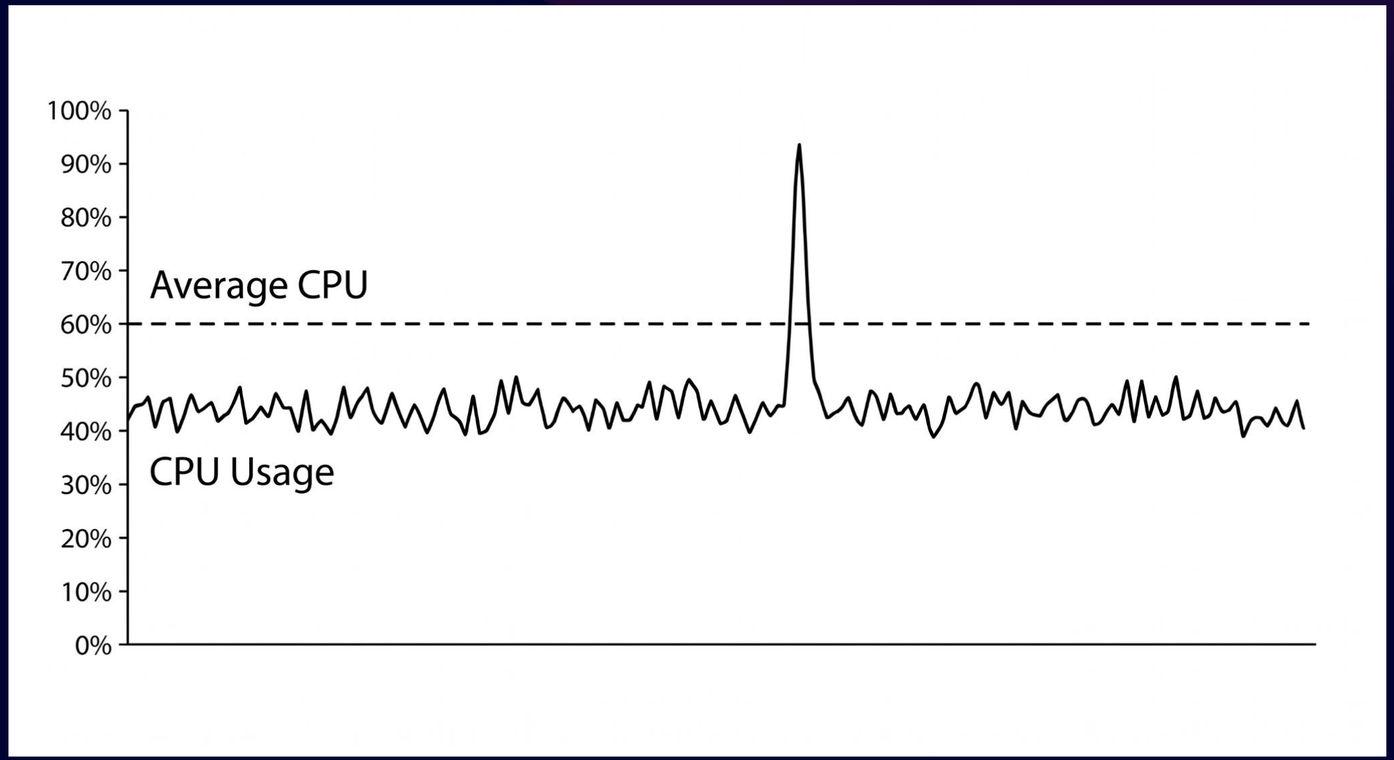
Consolidate

Provision, bin-pack, and remove node capacity.

The Flaws of Existing Autoscaling

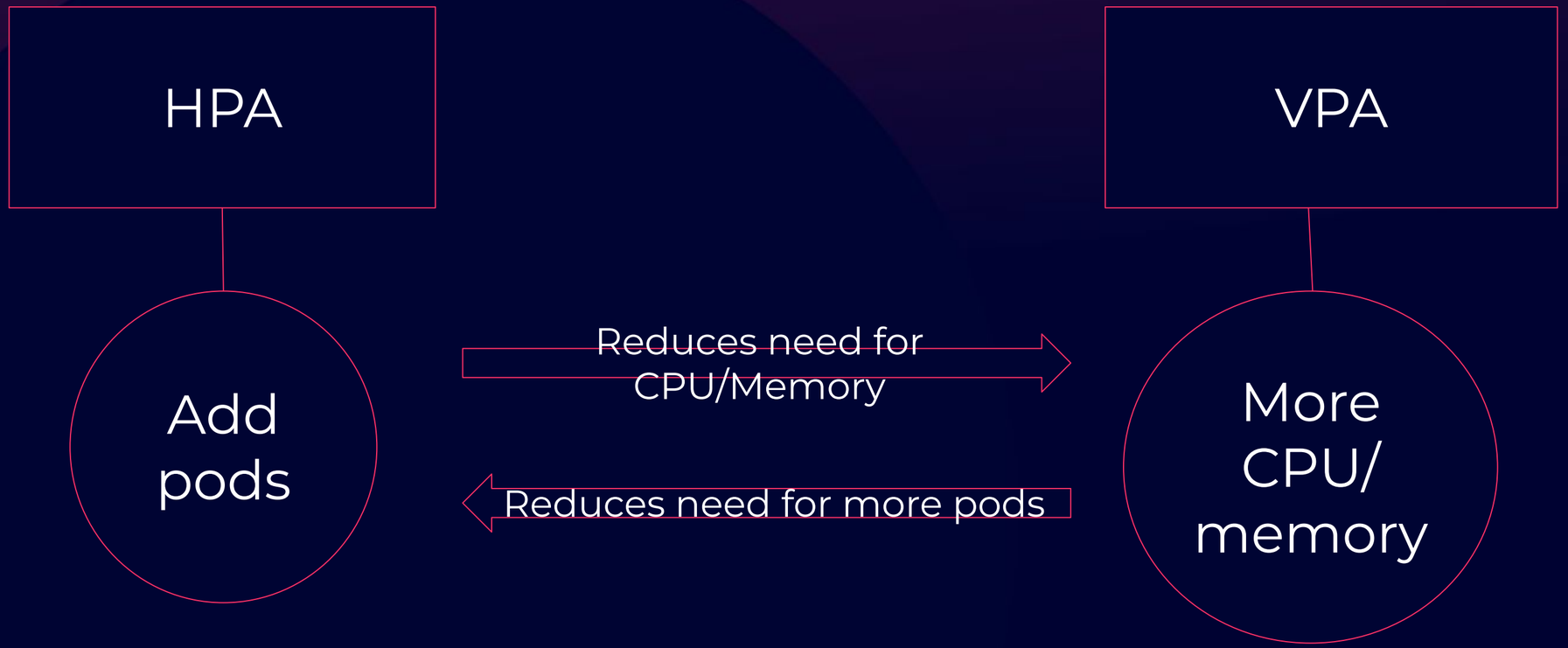
Signal Granularity: More or Less?

- CPU averages miss micro-bursts and complex performance profiles
- Leads to significantly delayed or incorrect decisions.



HPA/VPA Conflict

- HPA scales based on requested resources, while VPA modifies those requests.
- Uncoordinated, they issue contradictory instructions.



Eviction-Based Resizing

- Legacy VPA requires pod evictions to apply resource changes.
- Can disrupt cache-sensitive or stateful or slow-start workloads.

Layered Telemetry

Telemetry

- Workload under pressure?
- Pressure sustained or transient?
- Will a change improve service health or only change a dashboard number?

Layered Telemetry

Layer 0: Infrastructure

Layer : Workload

Layer 2: Application

Layer 3: K8s Change & risk

Layer 4: Decision & actuation

Infrastructure

- Capacity and resource pressure
- Measured at:
  - Node:
    - CPU, RAM
    - GPU with NVIDIA DCGM for scaling based on e.g., Streaming Multiprocessor utilization or memory bandwidth
  - Container:
    - cAdvisor: e.g., track micro-burst latency and avoid scaling on benign page cache use

Layer 0: Infrastructure

Layer : Workload

Layer 2: Application

Layer 3: K8s Change & risk

Layer 4: Decision & actuation

Workload

- Kubernetes configuration and lifecycle context
- Measuring:
  - Requests/limits
  - Replicas
  - Pending pods and HPA state
- Measured with, e.g.,
  - Kube-State-Metrics

Layer 0: Infrastructure

Layer 1: Workload

Layer 2: Application

Layer 3: K8s Change & risk

Layer 4: Decision & actuation

Application

- Why the workload is under pressure
- Kubernetes sees raw CPU or memory but doesn’t know, e.g., whether your Java app is choking on GC.
- Measured with
  - eBPF
    - Linux standard
    - Listens to the kernel
  - JVM uses eBPF through Coroot agent, e.g., for heap allocation rates and GC pressure
- Measuring, e.g.
  - Latency
  - Queue depth

Layer 0: Infrastructure

Layer 1: Workload

Layer 2: Application

Layer 3: K8s Change & risk

Layer 4: Decision & actuation

Change and Risk

- Whether an action is safe
- Tracking, e.g.,
  - Revision history
  - SLOs, policy
  - Disruption risk

Layer 0: Infrastructure

Layer 1: Workload

Layer 2: Application

Layer 3: K8s Change & risk

Layer 4: Decision & actuation

Decision and actuation

- What changes and how to apply changes
- Calculated with:
  - Unified logic
- Actuated with:
  - HPA
  - VPA Autoscaler
  - Cluster horizontal and vertical autoscaling
  - Bin-packing

Layer 0: Infrastructure

Layer 1: Workload

Layer 2: Application

Layer 3: K8s Change & risk

Layer 4: Decision & actuation

Mapping Signal to Decision

Mapping Signal to Decision (1)

- Container & Workload -> CPU/memory request and limit recommendations

Mapping Signal to Decision (2)

- Container & Workload -> CPU/memory request and limit recommendations
- **HPA Config & Replica History -> Scaling-context awareness**

Mapping Signal to Decision (3)

- Container & Workload -> CPU/memory request and limit recommendations
- HPA Config & Replica History -> Scaling-context awareness
- **Revision Timeline -> Distinguish app-change effects from optimizations**

Mapping Signal to Decision (4)

- Container & Workload -> CPU/memory request and limit recommendations
- HPA Config & Replica History -> Scaling-context awareness
- Revision Timeline -> Distinguish app-change effects from optimizations
- **GPU and JVM Signals -> Workload-specific bottleneck detection**

Mapping Signal to Decision (5)

- Container & Workload -> CPU/memory request and limit recommendations
- HPA Config & Replica History -> Scaling-context awareness
- Revision Timeline -> Distinguish app-change effects from optimizations
- GPU and JVM Signals -> Workload-specific bottleneck detection
- **Risk Indicators -> Prioritize resilience over savings actions**

In-Place Pod Resizing

- Stable from v1.35+
- Even with statelessness: Good for cold starts

Unified Scaling Logic

Avoid Conflict

- Safely actuate correct amount of Kubernetes resources
- By:
  - Harmonizing diverse telemetry signals
  - Via a centralized state machine
- With:
  - Custom Controller, e.g. Keda

Controller Operational Phases

Phase 1: Signal Aggregation
Phase 2: Axis Arbitration
Phase 3: Cooldown & Reconciliation

Always Impact-Aware

Cascade of Optimizations

Accurate pod requests → Better bin-packing → Fewer underutilized nodes → Autoscaler consolidation

Conclusions

Optimization is a Policy Decision

Savings  Headroom

How much latency is worth $100,000?

Principles

- Unify logic for different types of metrics and scaling.
- Stop scaling based on CPU/memory utilization; start scaling based on cost + service health.

Questions?

joshua@doit.com